

AI/ML-Based Adaptive Noise Cancellation for Defence Communications

Technical Approach & Jury Q&A Preparation Document

Problem Statement SIH 2026 - PS 26052	Team Phoenix
Organization DRDO / IDEX	Target Platform Raspberry Pi (edge / offline deployment)

1. Executive Summary

This project implements a lightweight, real-time speech-enhancement system that suppresses background and impulsive noise (e.g. gunfire, explosions, vehicle and equipment noise) from a speaker's voice before it is transmitted over a communications channel. The system runs entirely on-device on a Raspberry Pi with no cloud or network dependency, using a small recurrent neural network that predicts a per-frequency noise-suppression mask from streaming audio. The full pipeline - from raw microphone input to enhanced audio output - has been validated end-to-end: the trained model was exported to ONNX, deployed to a Raspberry Pi, and shown to run at roughly 60x faster than the real-time budget required for uninterrupted voice communication.

This document explains the technical approach in plain terms and compiles the questions the jury is likely to ask, organized by theme, with prepared answers grounded in what has actually been built and measured so far - and honest notes on what is still a work in progress rather than a finished claim.

2. Problem Statement & Motivation

Voice communication in defence and tactical environments is frequently degraded by background noise - gunfire, explosions, vehicle engines, wind, crowd chatter, and equipment noise - which reduces intelligibility over radio and other voice channels and can compromise both operational effectiveness and safety.

Traditional approaches fall into two categories, each with limitations relevant here:

- Physical/hardware ANC (active noise cancellation) headsets primarily cancel steady, low-frequency ambient noise through phase cancellation at the ear; they are far less effective against short, high-energy impulsive noise such as gunshots or explosions.
- Cloud-based AI noise suppression services (e.g. consumer call-cleanup tools) require continuous network connectivity and send audio off-device, which is unacceptable in a defence context where operational security and offline capability are essential.

This project addresses the gap: an AI-based, content-aware suppression system that works on both steady and impulsive noise, runs fully offline on low-cost embedded hardware, and adds minimal latency to the voice path.

3. Technical Architecture

3.1 Signal Processing Pipeline

Audio is processed at 16 kHz. The stream is split into overlapping analysis frames using a Short-Time Fourier Transform (STFT) with a 512-sample window (32 ms) and a 256-sample hop (16 ms), using a Hann window for smooth analysis/synthesis. This is the same frame size used both in training and in the real-time

deployment, so behaviour is consistent between the two. Enhanced audio is reconstructed with the inverse STFT and overlap-add, using window-sum normalization for artifact-free reconstruction.

3.2 Model Architecture

The core model is intentionally small: a single-layer GRU (64 hidden units) followed by a linear layer and a sigmoid activation, producing one suppression value (0 to 1) per frequency bin (257 bins, matching the 512-point FFT) for every audio frame. The model looks at a short rolling context of the 8 most recent frames (about 128 ms) of input magnitude spectra to make each prediction. This design was deliberately kept small and recurrent (rather than a larger CNN or Transformer-based architecture) for three reasons:

- Causal by construction - it only ever looks backward in time, which keeps end-to-end latency to essentially just the 16 ms frame hop, with no lookahead delay.
- Small footprint - the exported model is under 320 KB, comfortably fitting an embedded ARM device's memory and compute budget with no GPU or accelerator required.
- Fast enough to leave large real-time headroom even on modest hardware, which matters for reliability under field conditions rather than just best-case lab conditions.

3.3 Training Methodology

Training uses supervised, synthetic mixing: clean speech recordings are digitally mixed with real noise recordings at controlled signal-to-noise ratios, and the network is trained to predict a mask that, when applied to the noisy mixture's spectrum, reconstructs the clean speech. The noise set combines general ambient noise with noise samples specifically relevant to this problem statement's defence-communications context - including custom-recorded gunshot, explosion, and equipment/typing noise - so the model has direct exposure to the kinds of impulsive, high-energy sounds a purely generic noise-suppression model would not be tuned for.

Two tunable post-processing controls sit on top of the raw model output, without requiring retraining to adjust:

- Mask sharpening (gamma exponent) - controls how aggressively low-confidence (likely-noise) regions are pushed toward full suppression, letting the team dial suppression strength up or down based on listening tests.
- Automatic gain compensation - suppression by itself only ever removes energy, which would otherwise make the enhanced voice sound weaker than the original even when it is working correctly; a makeup-gain step restores the output to a natural, comparable loudness so the system is pleasant to actually use, not just measurably "quieter."

3.4 Real-Time Streaming Design

The deployed inference path processes audio strictly block-by-block (256 samples / 16 ms at a time) rather than operating on a full pre-recorded clip - this is the same causal architecture validated on a laptop for live microphone testing before being deployed to the Raspberry Pi, so the algorithm's real-time behaviour was proven before hardware deployment, not assumed.

3.5 Deployment Architecture

The trained PyTorch model is exported to the ONNX format (an open, framework-independent representation), which allows inference on the Raspberry Pi using the lightweight ONNX Runtime library instead of requiring a full PyTorch installation on constrained embedded hardware. The exported model's output was verified to match the original PyTorch model to within 1.64×10^{-7} - effectively bit-identical - confirming the conversion introduces no accuracy loss. On the Raspberry Pi, the pipeline runs entirely offline: microphone input arrives over USB, is processed on-device, and enhanced audio is played back over a standard audio output - no data ever leaves the device, and no network connection is required for the system to function.

4. Performance & Validation Results

- Model inference speed on Raspberry Pi (CPU-only, ONNX Runtime): 0.27 ms average, 1.52 ms maximum per 16 ms audio frame, measured over repeated inference passes - roughly 55-60x faster than the real-time budget, meaning the model is not the bottleneck for real-time performance even on modest embedded hardware.
- Exported model size: ~310 KB, keeping storage and load-time footprint minimal.
- Targeted suppression verified on real recordings: at the moment of a loud noise transient (e.g. a gunshot played near the microphone), measured peak amplitude reduction of roughly 55-79% and RMS energy reduction of roughly 50-89% in the enhanced output versus the raw input, with suppression consistently stronger during noise events than during quiet baseline periods - evidence the model is doing targeted, content-aware suppression rather than a blanket volume reduction.
- End-to-end pipeline validated across the full chain: training → PyTorch → ONNX export → Raspberry Pi deployment → on-device inference smoke test, all passing before any live-audio testing was attempted.

5. Key Design Decisions & Rationale

Decision	Rationale
Spectral masking vs. waveform-domain models	Lower compute cost, well-suited to real-time embedded inference; the established approach used by lightweight production noise-suppression systems (e.g. RNNoise-style architectures).
Small causal GRU vs. Transformer/CNN	Transformers and larger CNNs typically need more compute and/or non-causal context (lookahead), which adds latency; a small GRU meets the real-time and embedded-hardware constraints directly.
On-device (edge) inference vs. cloud	No network dependency in the field, no data leaves the device - directly aligned with defence operational security requirements, and removes latency/reliability risk from network conditions.
ONNX Runtime vs. full PyTorch on-device	Much smaller runtime footprint appropriate for an embedded ARM board; avoids installing and maintaining a full deep-learning framework on constrained hardware.
Tunable mask sharpening / AGC vs. fixed suppression	Lets the suppression strength and output loudness be tuned to real-world listening feedback without retraining the model.

6. Current Limitations & Roadmap

Presenting limitations proactively, with a clear plan, tends to land better with a technical jury than being caught off guard by the question. Current honest limitations:

- Requires a USB microphone/headset for live field use - the Raspberry Pi's built-in analog audio jack is output-only on every Pi model and cannot accept microphone input directly.
- Model generalization to noise types entirely absent from training data has not been formally benchmarked; behaviour is expected to degrade gracefully (it operates as a continuous spectral suppression, not closed-set classification) but this is not yet quantified.
- Multi-speaker / overlapping-talker scenarios have not been specifically validated - the system is tuned for single foreground speaker plus background noise.

- Environmental ruggedization (dust, shock, temperature extremes, MIL-STD-type qualification) has not been performed - the current build is a functional prototype on commodity development hardware, not a hardened field unit.

TEAM TO FILL IN: *exact current unit hardware cost, measured power draw, and any planned ruggedization/integration path with specific radio hardware, since these depend on choices your team has made that aren't captured in this document.*

7. Anticipated Jury Questions & Prepared Answers

Organized by theme. Answers are grounded in what has actually been built and measured; where a number depends on choices only your team has made, it's flagged rather than guessed.

7.1 Technical / Model Questions

Q: How does the model actually distinguish speech from noise?

A: It doesn't classify or recognize words - it works frame-by-frame in the frequency domain. For each of 257 frequency bins, the model predicts a value between 0 and 1 representing how much of that bin's energy, at that moment, is likely to be speech versus noise, based on patterns it learned from thousands of clean-speech-plus-noise training examples. Multiplying the noisy signal's spectrum by this mask suppresses the noise-dominated parts while preserving the speech-dominated parts.

Q: Why a GRU instead of a Transformer or CNN, which are more common in modern audio ML?

A: Latency and hardware budget. Transformers and many CNN-based denoisers either need more compute than a small embedded ARM CPU comfortably provides, or need to look ahead at future audio frames to make good predictions, which adds delay to the voice path - unacceptable for live communication. A small causal GRU only ever looks backward in time and is light enough that on our target hardware, measured inference time (0.27 ms) is about 55-60x faster than the 16 ms per-frame budget we need to hit.

Q: What is the true end-to-end latency of the system, not just the model?

A: The frame hop itself is 16 ms, which sets the processing granularity. Model inference adds a measured 0.27 ms on top of that. The remaining latency comes from the STFT/overlap-add reconstruction and standard audio I/O buffering, which we have not benchmarked in isolation yet - we'd recommend measuring end-to-end mic-to-headphone latency directly with a timing test before quoting a single combined number to the jury.

Q: How was the model trained - what data, and how much?

A: Supervised training on synthetically mixed audio: clean speech clips mixed with real noise recordings at controlled signal-to-noise ratios, so the model always has a known "ground truth" clean target to learn to reconstruct. The noise set was extended specifically for this problem statement with custom-recorded gunshot, explosion, and equipment noise, in addition to general ambient noise, so the model has direct exposure to the noise types most relevant to a defence-communications scenario.

Q: Why export to ONNX rather than running the trained PyTorch model directly on the Pi?

A: PyTorch's full runtime is heavy for an embedded board - larger install footprint and more dependencies than an inference-only device needs. ONNX Runtime is a lightweight, purpose-built inference engine with official ARM64 builds for Raspberry Pi OS. We verified the exported model's output matches the original PyTorch model to within 1.64×10^{-7} - so this is a deployment optimization with no measurable accuracy cost.

Q: Does the system ever suppress real speech by mistake?

A: It can, in principle, like any statistical suppression system - this is managed rather than eliminated, through a tunable sharpening control that lets us trade off suppression strength against speech-fidelity risk

based on real listening tests, rather than a single fixed hardcoded setting. In our own listening validation, speech remained audible and intelligible through the enhanced output even as noise transients were suppressed by 70-90% in peak amplitude.

Q: What happens with a noise type the model has never seen in training?

A: Because the model operates as a continuous, per-frequency-bin suppression rather than recognizing specific noise categories, we expect it to generalize reasonably to acoustically similar unseen noise, with performance degrading gracefully rather than failing outright on truly novel noise. That said, this has not been formally benchmarked against a held-out set of completely unseen noise types, and we'd flag that as honest, ongoing work rather than a solved problem.

7.2 Feasibility, Viability & Deployment

Q: What hardware does this actually run on, and does it need special equipment?

A: A Raspberry Pi (or comparable low-cost single-board computer) plus a standard USB microphone/headset for audio input and output - no GPU, no accelerator, and no cloud connection required. The Pi's own built-in audio jack cannot accept microphone input on any model, which is why USB audio hardware is a requirement, not just a convenience.

TEAM TO FILL IN: *exact unit cost based on the specific Pi model and USB audio hardware your team used, since pricing varies and wasn't tracked as part of the model/software work.*

Q: Does the system need internet or network access to function?

A: No. Once deployed, the entire pipeline - capturing audio, running the model, and producing enhanced output - happens on-device with no network calls at all. Network access was only ever used during our own development process (for remote file transfer and setup), never as part of the deployed system's actual operation. This is an important point for a defence context: no audio or data ever leaves the device.

Q: What is the power consumption, and could this run on battery in the field?

A: Single-board computers like the Raspberry Pi are designed for low power draw relative to general-purpose PCs and are commonly battery/portable-power operated in embedded projects, but we have not yet benchmarked our specific setup's actual power draw under sustained inference load.

TEAM TO FILL IN: *a measured power draw figure (e.g. with a USB power meter) under real inference load before presenting - this is a quick, high-value measurement to have ready.*

Q: How would this scale to equip many units, not just one prototype?

A: The trained model is a single small file (roughly 310 KB) that can be copied identically to any number of edge devices - there is no per-unit cloud cost, licensing server, or centralized dependency, since every unit runs fully independently and offline. Manufacturing/procurement scaling of the hardware itself follows standard embedded-device supply chains rather than anything specific to this software.

Q: How does this compare to existing noise-cancellation approaches - hardware ANC headsets, or tools like RNNoise/Krisp?

A: Hardware ANC headsets work through physical phase-cancellation of ambient sound at the ear, which is effective mainly against steady, low-frequency noise (engine hum, wind) and considerably less effective against short, high-energy impulsive noise such as gunfire or explosions - exactly the noise types most relevant here. Software tools like RNNoise use a broadly similar spectral-masking concept to ours, but are generally trained and tuned for general-purpose voice-call noise (typing, fans, traffic) rather than the specific tactical/impulsive noise profile this problem statement targets, and typically assume a networked call context rather than a fully offline embedded deployment.

7.3 Field-Readiness & Limitations

Q: Is this ready for actual field deployment today?

A: It's a validated functional prototype, not yet a hardened field unit. The AI pipeline itself - model, real-time processing, offline operation - is built and tested end-to-end. What remains before field deployment is hardware ruggedization (enclosure, shock/dust/temperature resilience), integration with actual radio/communications equipment, and formal environmental qualification testing, none of which are part of the software/ML scope demonstrated here.

Q: How would this integrate with existing military communications equipment?

A: The system takes standard microphone input and produces enhanced audio output over USB audio, so conceptually it sits inline between a user's headset/mic and the radio unit, cleaning the voice signal before it is transmitted. The specific physical/electrical integration path with a given radio system would be a hardware-integration exercise beyond the current prototype's scope.

Q: What don't you know yet / what would you do with more time?

A: Formally benchmark generalization to noise types outside the training set; measure true end-to-end latency and power draw under load; test with multiple simultaneous speakers; and move from a development prototype toward a ruggedized, radio-integrated field unit. We think naming these directly, rather than overclaiming readiness, is the stronger answer to give a technical jury.

7.4 Wireless Communication, Range & Two-Way Operation

Q: Today's demo has a wired USB mic and wired earphones with the speaker and listener standing right next to each other – how does this become usable at real combat distances where two people can't be physically wired together?

A: We are not building a replacement for wireless communication – soldiers already carry radios or tactical headsets for the actual over-distance link, and re-engineering that is not the problem this project targets. Our system is a small AI processing stage designed to sit inline with that existing signal chain: on the transmitting side, it cleans the speaker's voice before it enters the radio; on the receiving side, it can equally clean up what comes out of the radio before it reaches the listener. Today's prototype is deliberately wired and co-located because that isolates the one variable we actually needed to prove – that the AI can denoise gunfire-grade noise in real time on lightweight embedded hardware. Radios already solve the distance/wireless problem; we did not need to re-solve it to demonstrate our contribution.

Q: Have you tested this integrated with an actual radio or wireless communications device?

A: Not yet – that is the next integration step, and it is a hardware-availability question (getting access to a tactical radio or comparable equipment to test with) rather than an open algorithm question. The processing itself is already proven to run well within the real-time budget on embedded hardware; wiring it into a specific radio's audio path is systems integration, not unsolved research.

Q: How much additional delay would your processing add on top of whatever delay the radio link itself introduces?

A: The frame hop is 16 ms, and measured model inference adds only 0.27 ms on top of that (about 55-60x faster than the real-time budget) – so our contribution to end-to-end delay is on the order of tens of milliseconds at most, small relative to typical radio or network latency, and not perceptible as lag in normal conversation.

Q: Today's unit only has one microphone and one earphone – doesn't that mean you've only solved noise cleanup for one side of a conversation, not both directions?

A: That's a fair and precise reading of today's demo: it shows one direction – cleanup at the point where a voice is captured, before it would be transmitted. The model itself, though, doesn't know or care where its noisy input comes from; it takes in noisy audio and outputs cleaned audio regardless of source. A fielded two-way system would place a second, identical, unmodified copy of this same pipeline on the receiving side, taking its input from the radio's received audio instead of a live microphone. That's the same code and the same trained model – there is no new engineering question there, just another instance of what is already demonstrated working.

Q: If it generalizes that easily, why not just build and demo the two-way version now?

A: Given the time available before presenting, we made a deliberate scope decision: prove the hard, novel part – real-time AI noise suppression on embedded hardware – as solidly as possible, rather than spend that time re-implementing wireless audio transport, which is a solved problem already handled by existing radio hardware and not something this project needs to reinvent to be valuable.

Q: Would this replace a soldier's earpiece or radio?

A: No – it's designed to supplement existing gear as an inline processing module, not replace the radio or headset a soldier already carries.

TEAM TO FILL IN: *an actual test with a real tactical radio or comparable two-way system, and a timeline for that integration, since this depends on hardware access your team may or may not have secured yet.*

8. How to Use This Document

Treat the numbers in this document as ground truth from what has actually been built and measured so far - they came directly from the project's own test runs, not estimates. The boxed "TEAM TO FILL IN" notes mark the handful of places (unit cost, power draw, ruggedization plans) that depend on decisions or measurements only your team has, and are worth nailing down before presentation day so you're not caught improvising a number on the spot.